

-- coding: utf-8 --

""" 鱼书 第4–6章 参考实现（带注释，供学习）

用法建议：

1. 从上往下读，重点啃每个层的 backward 注释；
2. 直接 `python ref.py` 运行，先看"梯度确认"是否通过、再看训练 loss 下降；
3. 看懂后【合上本文件】，自己默写复现一遍，这才是真正的目标。只依赖 numpy，不需要 MNIST。"""

```
import numpy as np from collections import OrderedDict
```

```
=====
=====
```

第4章：损失函数 与 数值梯度

```
=====
=====
```

```
def mean_squared_error(y, t): """均方误差。y: 预测, t: 真实(one-hot)。""" return 0.5 *
np.sum((y - t) ** 2)
```

```
def cross_entropy_error(y, t): """交叉熵误差（batch 版，同时支持 one-hot 和"标签索引"两种
t）。
```

```
y: (N, C) 预测概率
t: (N, C) one-hot 或 (N,) 整数标签索引
"""
if y.ndim == 1:                                # 单样本时补成 batch=1
    t = t.reshape(1, t.size)
    y = y.reshape(1, y.size)

if t.size == y.size:                            # t 是 one-hot -> 转成标签索引
    t = t.argmax(axis=1)

batch_size = y.shape[0]
delta = 1e-7                                # 防止 log(0)
```

```

delta = 1e-4 # 防止 log(0)
# 只取每个样本"正确类别"对应的预测概率，取 log 后求和
return -np.sum(np.log(y[np.arange(batch_size), t] + delta)) / batch_size

```

```

def numerical_gradient(f, x): """对 x 的每个元素求数值梯度（中心差分）。f: 接受 x、返回标
量的函数；x: 任意形状 ndarray。注意：这个函数慢得可怕，但绝对可信——第5章拿它当梯
度确认的"标尺"。""" h = 1e-4 grad = np.zeros_like(x) it = np.nditer(x, flags=['multi_index'],
op_flags=['readwrite']) while not it.finished: idx = it.multi_index tmp = x[idx] x[idx] = tmp + h
fxh1 = f(x) # f(x+h) x[idx] = tmp - h fxh2 = f(x) # f(x-h) grad[idx] = (fxh1 - fxh2) / (2 * h) x[idx]
= tmp # 还原，别忘了 it.iternext() return grad

```

```

=====
=====

```

第5章：各种"层"（每层都实现 forward / backward）

```

=====
=====

```

```

def softmax(x): """softmax，减去最大值防溢出。支持 (N,C) 和 (C,)。""" if x.ndim == 2: x = x -
x.max(axis=1, keepdims=True) x = np.exp(x) return x / x.sum(axis=1, keepdims=True) x = x -
np.max(x) return np.exp(x) / np.sum(np.exp(x))

```

```

class Relu: def init(self): self.mask = None # 记录前向时 x<=0 的位置

```

```

def forward(self, x):
    self.mask = (x <= 0)
    out = x.copy()
    out[self.mask] = 0 # 负的置 0，正的原样透传
    return out

def backward(self, dout):
    # 前向被"堵住"(x<=0)的位置，梯度也堵成 0；其余原样回传
    dout[self.mask] = 0
    return dout

```

```

class Sigmoid: def init(self): self.out = None # 缓存前向输出 y，backward 要用

```

```

def forward(self, x):
    self.out = 1 / (1 + np.exp(-x))

```

```
return self.out
```

```
def backward(self, dout):  
    #  $d\sigma/dx = y(1-y)$ , 只用前向输出  $y$  即可, 很优雅  
    return dout * self.out * (1.0 - self.out)
```

```
class Affine: """全连接层:  $Y = X \cdot W + b$ """  
    def init(self, W, b):  
        self.W = W  
        self.b = b  
        self.x = None  
        # 缓存输入 X, backward 求 dW 要用  
        self.dW = None  
        self.db = None
```

```
def forward(self, x):  
    self.x = x  
    return np.dot(x, self.W) + self.b  
  
def backward(self, dout):  
    # 关键三式, 注意形状对账:  
    dx = np.dot(dout, self.W.T)      #  $dX = dout \cdot W^T$     -> 形状同 X  
    self.dW = np.dot(self.x.T, dout) #  $dW = X^T \cdot dout$     -> 形状同 W  
    self.db = np.sum(dout, axis=0)    # db 沿 batch 维求和 -> 形状同 b  
    return dx                        # 把对输入的梯度往下游传
```

```
class SoftmaxWithLoss: """softmax + 交叉熵 合成一层 (最后一层)"""  
    def init(self):  
        self.loss = None  
        self.y = None  
        # softmax 输出  
        self.t = None  
        # 真实标签
```

```
def forward(self, x, t):  
    self.t = t  
    self.y = softmax(x)  
    self.loss = cross_entropy_error(self.y, self.t)  
    return self.loss  
  
def backward(self, dout=1):  
    # 反向结果干净得惊人:  $(y - t) / batch\_size$   
    batch_size = self.t.shape[0]  
    if self.t.size == self.y.size:      # t 是 one-hot  
        dx = (self.y - self.t) / batch_size  
    else:                                # t 是标签索引  
        dx = self.y.copy()  
        dx[np.arange(batch_size), self.t] -= 1  
        dx = dx / batch_size  
    return dx
```

=====

=====

第5章：用层组装两层网络 + 梯度确认

=====

=====

```
class TwoLayerNet: def init(self, input_size, hidden_size, output_size, weight_init_std=0.01): #
---- 参数初始化（注意：W 用随机值打破对称，b 用 0 没问题）---- self.params = {}
self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
self.params['b1'] = np.zeros(hidden_size) self.params['W2'] = weight_init_std *
np.random.randn(hidden_size, output_size) self.params['b2'] = np.zeros(output_size)
```

```
# ---- 用 OrderedDict 按顺序堆层 ----
self.layers = OrderedDict()
self.layers['Affine1'] = Affine(self.params['W1'], self.params['b1'])
self.layers['Relu1'] = Relu()
self.layers['Affine2'] = Affine(self.params['W2'], self.params['b2'])
self.lastLayer = SoftmaxWithLoss()
```

```
def predict(self, x):
    for layer in self.layers.values():          # 逐层 forward
        x = layer.forward(x)
    return x
```

```
def loss(self, x, t):
    y = self.predict(x)
    return self.lastLayer.forward(y, t)
```

```
def accuracy(self, x, t):
    y = np.argmax(self.predict(x), axis=1)
    if t.ndim != 1:
        t = np.argmax(t, axis=1)
    return np.sum(y == t) / float(x.shape[0])
```

```
def numerical_gradient(self, x, t):
    """慢但可信：用数值微分对每个参数求梯度（仅用于梯度确认）"""
    loss_W = lambda W: self.loss(x, t)
    grads = {}
    for key in ('W1', 'b1', 'W2', 'b2'):
        grads[key] = numerical_gradient(loss_W, self.params[key])
    return grads
```

```
def gradient(self, x, t):
    """反向传播：高效求所有梯度（实际使用的就是这个）"""
    self.loss(x, t) # 先 forward（顺便缓存中间值）
    dout = self.lastLayer.backward(1) # 从最后一层开始反向
    layers = list(self.layers.values())
    layers.reverse() # 逆序逐层 backward
    for layer in layers:
        dout = layer.backward(dout)
    grads = { # 收集各 Affine 层算好的梯度
```

```

        'W1': self.layers['Affine1'].dW,
        'b1': self.layers['Affine1'].db,
        'W2': self.layers['Affine2'].dW,
        'b2': self.layers['Affine2'].db,
    }
    return grads

```

=====

=====

第6章：优化器（参数更新）

=====

=====

```
class SGD: def init(self, lr=0.01): self.lr = lr
```

```

def update(self, params, grads):
    for key in params.keys():
        params[key] -= self.lr * grads[key]    #  $W \leftarrow W - \eta \cdot g$ 

```

```
class Momentum: def init(self, lr=0.01, momentum=0.9): self.lr = lr self.momentum = momentum self.v = None # 速度
```

```

def update(self, params, grads):
    if self.v is None:
        self.v = {k: np.zeros_like(v) for k, v in params.items()}
    for key in params.keys():
        self.v[key] = self.momentum * self.v[key] - self.lr * grads[key]
        params[key] += self.v[key]

```

```
class AdaGrad: def init(self, lr=0.01): self.lr = lr self.h = None # 历史梯度平方累积
```

```

def update(self, params, grads):
    if self.h is None:
        self.h = {k: np.zeros_like(v) for k, v in params.items()}
    for key in params.keys():
        self.h[key] += grads[key] * grads[key]
        params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)

```

```
class Adam: """Momentum(一阶矩 m) + AdaGrad(二阶矩 v) + 偏差修正 当今默认首选 """
```



```
class Adam:
    momentum(一阶矩 m) + AdaGrad(二阶矩 v) + 偏差修正。 3个参数自底。
def init(self, lr=0.001, beta1=0.9, beta2=0.999): self.lr = lr self.beta1 = beta1 self.beta2 =
beta2 self.iter = 0 self.m = None self.v = None
```

```
def update(self, params, grads):
    if self.m is None:
        self.m = {k: np.zeros_like(v) for k, v in params.items()}
        self.v = {k: np.zeros_like(v) for k, v in params.items()}
    self.iter += 1
    # 偏差修正: 早期 m、v 偏小, 用这个系数把学习率放大回来
    lr_t = self.lr * np.sqrt(1.0 - self.beta2 ** self.iter) / (1.0 - self.beta1 ** self.it
    for key in params.keys():
        self.m[key] += (1 - self.beta1) * (grads[key] - self.m[key])      # 一阶矩
        self.v[key] += (1 - self.beta2) * (grads[key] ** 2 - self.v[key]) # 二阶矩
        params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)
```

=====

=====

Demo: 运行验证

=====

=====

```
if name == "main": np.random.seed(0)
```

```
# ----- (1) 梯度确认: 数值微分 vs 反向传播 -----
print("=== 梯度确认 (两者应几乎一致) ===")
net = TwoLayerNet(input_size=10, hidden_size=5, output_size=3)
x = np.random.randn(4, 10)
t = np.zeros((4, 3))
t[np.arange(4), np.random.randint(0, 3, 4)] = 1      # one-hot 标签

g_num = net.numerical_gradient(x, t)
g_bp = net.gradient(x, t)
for key in g_num:
    diff = np.average(np.abs(g_bp[key] - g_num[key]))
    print(f" {key}: 平均差 = {diff:.3e}")

# ----- (2) 训练循环: 看 loss 下降、acc 上升 -----
print("\n=== 训练循环 (mini-batch + Adam) ===")
N = 300
X = np.random.randn(N, 10)
```

```

W_true = np.random.randn(10, 3)
T_idx = np.argmax(X @ W_true, axis=1)          # 造一个可分的 3 分类任务
T = np.zeros((N, 3))
T[np.arange(N), T_idx] = 1

net2 = TwoLayerNet(input_size=10, hidden_size=20, output_size=3)
opt = Adam(lr=0.01)
for step in range(1, 1001):
    idx = np.random.choice(N, 32)              # 取 mini-batch
    grads = net2.gradient(X[idx], T[idx])      # 反向传播算梯度
    opt.update(net2.params, grads)             # 更新参数
    if step % 200 == 0:
        print(f"  step {step:4d}  loss={net2.loss(X, T):.4f}  acc={net2.accuracy(X, T):.3f}

```